Course End Project

FIRE DETECTION AND ALERT SYSTEM USING ARDUINO

IOT LABORATORY (A8603)

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

24881A05F2 INDLA BHARATH REDDY

24881A05F6 KANTHA SHARVAN KUMAR

Under the Guidance of

MS BAALNE ANJALI

Assistant Professor, Dept of CSE

Department of Computer Science and Engineering

VARDHAMAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC with A++ Grade,

UGC "College with Potential for Excellence" and ISO 9001:2015 Certified

Shamshabad – 501 218, Hyderabad

2025 - 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to JNTUH,
Approved by AICTE, Accredited by NAAC with A++
Grade Kacharam, Shamshabad, Hyderabad- 501
218, Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Fire Detection And Alert System Using Arduino"** is carried out by **I.BHARATH REDDY** and **K.SHARVAN KUMAR** with Roll Numbers **24881A05F2 and 24881A05F6** towards **A8603 – IOT Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty	**Signature of the HOD**
Ms. Baalne Anjali Assistant Professor, CSE	Dr. Karnati Ramesh HOD, CSE

VARDHAMAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC with A++ Grade,
UGC "College with Potential for Excellence" and ISO 9001:2015 Certified

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course End Project

IOT Laboratory (A8603)

Check List

S. No.	Content	Status (✓ / ✗)
1	Abstract	✓
2	Objectives	✓
3	Problem Statement	✓
4	Algorithm	✓
5	Flow Chart	✓
6	Source Code	✓
7	Input	✓
8	Output	✓
9	Conclusion	✓
10	Future Scope	✓
11	References	✓

Signature of the Guide
Ms. Baalne Anjali
Assistant Professor
Dept. of CSE

Course End Project

IOT Laboratory (A8603)

Table of Contents

S. No.	Content	Page No.
1	Abstract	5
2	Problem Statement	5
3	Objectives	6
4	Algorithm	6
5	Flow Chart	7
6	Source Code	8
7	Input	11
8	Output	12
9	Conclusion	13
10	Future Scope	13
11	References	14

1. ABSTRACT

The Fire Detection and Alert System is an Arduino-based embedded system designed to detect the presence of fire, flame, and smoke in indoor environments and trigger immediate audio-visual alerts. Conventional smoke detectors are passive and limited in their ability to provide real-time status feedback. This project addresses that limitation by implementing a multi-sensor detection system using an IR Flame Sensor and an MQ-2 Gas/Smoke Sensor, both interfaced with an Arduino Uno microcontroller.

When either sensor detects fire or smoke beyond a defined threshold, the Arduino activates a buzzer for an audible alarm, illuminates a red LED as a visual indicator, and displays a warning message on a 16×2 LCD screen. The system continuously monitors sensor values in a polling loop and responds within milliseconds of detection, providing a fast and reliable safety response.

Once a fire is detected, the system automatically triggers an alarm (buzzer or siren) to alert nearby people. Additionally, it sends notifications through communication modules like GSM or Wi-Fi to inform users remotely via SMS or mobile applications. The system can also activate safety mechanisms such as water sprinklers or exhaust fans to control the fire.

The system operates in a continuous loop, ensuring real-time monitoring and quick response. It is reliable, cost-effective, and suitable for homes, industries, and public places. By providing early detection and rapid alerts, the system helps in minimizing risks, protecting property, and saving lives.

This project demonstrates fundamental IoT and embedded systems concepts including digital and analog sensor interfacing, threshold-based decision logic, LCD communication via I2C, and real-time event-driven programming. It serves as a practical, low-cost prototype for smart fire safety systems applicable to homes, laboratories, and industrial environments.

2. PROBLEM STATEMENT

Fire accidents are among the most destructive and life-threatening events that occur in residential, commercial, and industrial settings. A large number of fire-related casualties and property losses occur because fire is not detected early enough to allow safe evacuation or timely intervention. Traditional fire safety systems rely on single-sensor smoke detectors that may produce false negatives in the early stages of a fire or false positives due to cooking smoke or steam.

Furthermore, most basic detectors provide only an audible alarm with no visual display of the type of threat or the current sensor status, making it difficult to assess the severity of the situation remotely. In resource-constrained deployments such as small offices, hostels, server rooms, or IoT-based smart home systems, a cost-effective, multi-sensor detection mechanism with clear status feedback is essential.

Fire accidents pose a serious threat to human life, property, and the environment. In many homes, industries, and public places, fire incidents are often detected too late due to the absence of efficient monitoring systems or reliance on manual detection. This delay in identifying fire hazards can lead to severe damage, financial loss, and even loss of lives.

Traditional fire safety methods, such as standalone alarms, may not provide real-time alerts to users who are away from the location. Additionally, lack of integration between detection, alert, and response systems reduces the effectiveness of fire prevention measures.

Therefore, the problem is: How can a low-cost, microcontroller-based system be designed to detect both flame and smoke simultaneously, provide immediate audio-visual alerts, and display real-time status information, thereby improving early fire detection and response time?

This project proposes a solution using an Arduino Uno microcontroller, an IR Flame Sensor, an MQ-2 Gas Sensor, a buzzer, LEDs, and a 16×2 LCD to build a reliable fire detection and alert system.

3. OBJECTIVES

The primary objectives of this project are:

- To design and implement a multi-sensor fire detection system using an Arduino Uno microcontroller.
- To interface an IR Flame Sensor for detecting the presence of open flame using infrared radiation.
- To interface an MQ-2 Gas/Smoke Sensor for detecting smoke and combustible gases such as LPG and propane.
- To activate a buzzer and red LED as immediate audio-visual alert outputs upon fire/smoke detection.
- To display real-time system status and alert messages on a 16×2 LCD display using I2C communication.
- To implement a threshold-based decision algorithm that minimises false alarms from minor sensor fluctuations.
- To develop practical skills in sensor interfacing, digital/analog I/O, LCD programming, and embedded C on Arduino.

4. ALGORITHM

The algorithm governing the fire detection system's operation is as follows:

Step 1: Initialization

Initialize the Arduino Uno. Set digital pin for the flame sensor as INPUT. Set pins for buzzer and LED as OUTPUT. Initialize the I2C LCD (address 0x27). Display 'System Ready' on the LCD. Define smoke threshold value (e.g., 300 analog units).

Step 2: Sensor Reading Loop

Read the digital output of the Flame Sensor (flameVal = digitalRead(flamePin)). Read the analog output of the MQ-2 Smoke Sensor (smokeVal = analogRead(smokePin)).

Step 3: Flame Detection Check

If flameVal == LOW (flame detected): Activate buzzer (digitalWrite(buzzerPin, HIGH)). Turn on red LED. Display 'FIRE DETECTED!' on LCD line 1. Display 'EVACUATE NOW!' on LCD line 2.

Step 4: Smoke Detection Check

Else if smokeVal > threshold: Activate buzzer. Turn on red LED. Display 'SMOKE DETECTED!' on LCD line 1. Display 'Check Area!' on LCD line 2.

Step 5: Normal Condition

Else: Deactivate buzzer (digitalWrite(buzzerPin, LOW)). Turn off LED. Display 'System Normal' on LCD line 1. Display 'No Fire/Smoke' on LCD line 2.

Step 6: Loop

Add a small delay (200 ms) for sensor stabilisation. Repeat from Step 2 continuously.

5. PROPOSED SYSTEM

The proposed system consists of an Arduino Uno microcontroller as the central processing unit, an IR Flame Sensor and MQ-2 Gas Sensor for multi-modal fire detection, a buzzer and LED for alert outputs, and a 16×2 I2C LCD for real-time status display. The system operates in a continuous polling loop and responds within milliseconds to fire or smoke events.

5.1 Components Used

S. No.	Component	Quantity	Description
1	Arduino Uno	1	Microcontroller board (ATmega328P)
2	Flame Sensor (IR)	2	Detects presence of fire or open flame
3	MQ-2 Gas Sensor	1	Detects smoke and combustible gases
4	Buzzer (Active)	1	Audible alert on fire detection
5	LED (Red)	2	Visual indicator for fire/alarm
6	16×2 LCD Display	1	Displays system status and warnings
7	I2C Module (PCF8574)	1	Reduces LCD wiring to 2 pins
8	Resistors (220 Ω)	2	Current-limiting resistors for LEDs
9	Breadboard	1	For circuit prototyping
10	Jumper Wires	~20	For making circuit connections
11	USB Cable	1	For programming and power supply

5.2 Circuit Description

The IR Flame Sensor's digital output pin is connected to digital pin 7 of the Arduino. The sensor outputs LOW (logic 0) when a flame is detected within its sensing range (60 cm, 60° angle) and HIGH under normal conditions. The sensor is powered from the Arduino's 5 V and GND rails.

The MQ-2 Gas Sensor's analog output (AO) pin is connected to analog pin A0 of the Arduino. The sensor produces a voltage proportional to the concentration of smoke or gas. The Arduino's 10-bit ADC maps this to a value between 0 and 1023. A threshold of 300 analog units is used to distinguish normal air from hazardous smoke levels.

The active buzzer is connected to digital pin 8. The red LED, in series with a 220 Ω current-limiting resistor, is connected to digital pin 9. The 16×2 LCD is connected via an I2C module (PCF8574) to the Arduino's SDA (A4) and SCL (A5) pins, reducing the wiring from 6 data lines to just 2.

5.3 Flow Chart

The flowchart begins with system initialization and LCD welcome display. The main loop continuously reads both sensor values. If the flame sensor triggers (LOW), the alarm sequence is activated immediately. Otherwise, the smoke sensor value is compared to the threshold. If exceeded, the alarm activates. When neither condition is met, the system displays 'Normal' status. The loop repeats after a short delay.

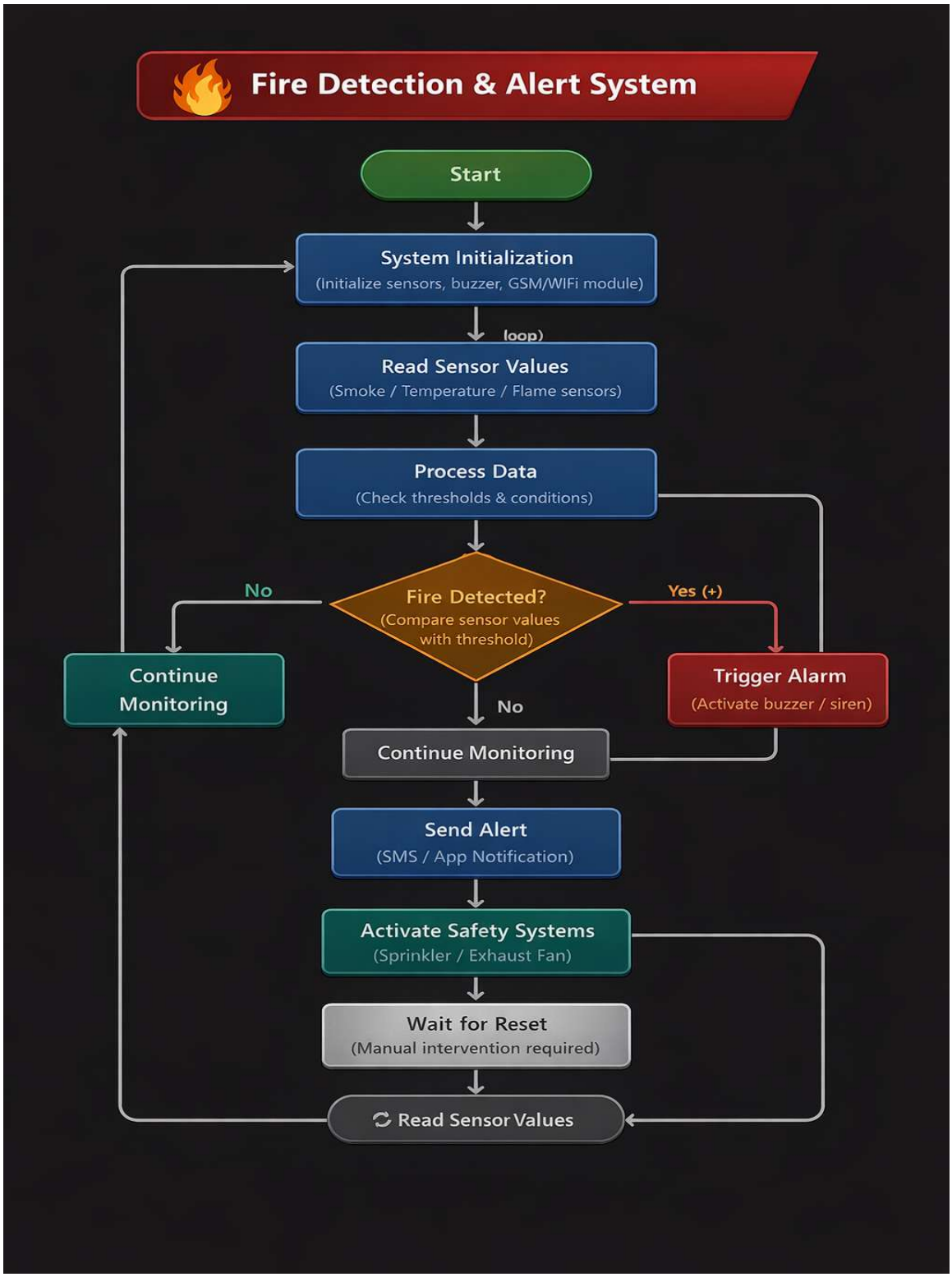

Figure 1: Flowchart of Fire Detection and Alert System

5.4 Working Model

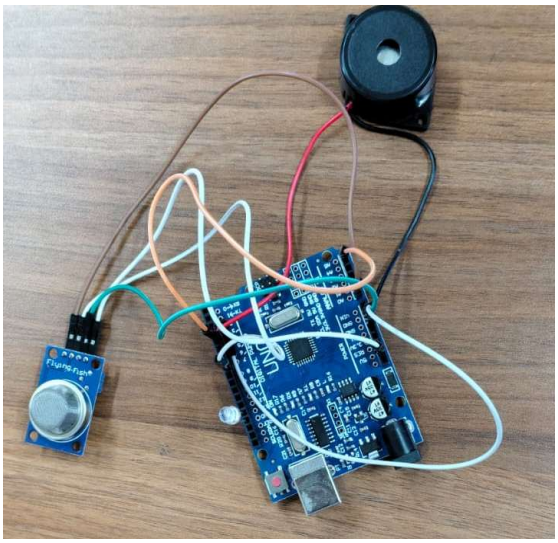

Figure 2: Hardware Circuit Diagram / Working Model

The hardware model consists of the Arduino Uno on a breadboard with the IR Flame Sensor and MQ-2 Sensor positioned to face the environment. The buzzer and red LED are mounted visibly on the breadboard for demonstration. The 16×2 LCD is connected via I2C and placed prominently to display system status.

When a lighter flame or a burning match is brought near the IR sensor, the system immediately triggers the buzzer and LED while the LCD displays 'FIRE DETECTED! EVACUATE NOW!'. Similarly, when smoke from a candle or match is directed toward the MQ-2 sensor, the analog value exceeds the threshold and the same alarm sequence is activated. In clean-air conditions the system displays 'System Normal / No Fire/Smoke'.

6. SOURCE CODE

```
// Fire Detection System using Arduino Uno + MQ-2 Sensor
// Components: MQ-2 sensor, buzzer, red LED, green LED
// MQ-2 Connections:
//   VCC  → 5V
//   GND  → GND
//   AOUT → A0
//   DOUT → D7

const int mq2Pin     = A0;  // MQ-2 analog output
const int mq2Digital = 7;   // MQ-2 digital output (DOUT)
const int buzzerPin  = 3;   // Active buzzer
const int redLED     = 4;   // Red LED (alarm)
const int greenLED   = 13;   // Green LED (safe)

const int threshold  = 400; // Analog threshold (0–1023)

void setup() {
  pinMode(buzzerPin,  OUTPUT);
  pinMode(redLED,     OUTPUT);
  pinMode(greenLED,   OUTPUT);
  pinMode(mq2Digital, INPUT);   // DOUT pin as input
  Serial.begin(9600);

  // Warm-up time: MQ-2 needs ~30 sec to stabilise
  Serial.println("Warming up MQ-2 sensor... please wait 30 seconds.");
  for (int i = 10; i > 0; i--) {
    Serial.print(i);
    Serial.println("s remaining...");
    digitalWrite(greenLED, HIGH);
    delay(500);
    digitalWrite(greenLED, LOW);
    delay(500);
  }
  Serial.println("System ready!");
  Serial.println("----------------------------------------");
}

void loop() {
  int analogValue  = analogRead(mq2Pin);
  int digitalValue = digitalRead(mq2Digital);

  // Digital pin is active LOW on most MQ-2 modules
  // LOW  = smoke/gas detected (DOUT pulls low)
  // HIGH = clean air
  bool digitalAlarm = (digitalValue == LOW);
  bool analogAlarm  = (analogValue > threshold);

  // Print both readings to Serial Monitor
  Serial.print("Analog: ");
```

```
  Serial.print(analogValue);
  Serial.print(" / 1023   |   Digital: ");
  Serial.print(digitalValue);
  Serial.print("   |   Analog status: ");
  Serial.print(analogAlarm  ? "SMOKE" : "Clean");
  Serial.print("   |   Digital status: ");
  Serial.println(digitalAlarm ? "SMOKE DETECTED!" : "Clean air");

  // ALARM if EITHER analog OR digital triggers
  if (analogAlarm || digitalAlarm) {
    Serial.println("*** FIRE / SMOKE DETECTED! ***");
    digitalWrite(redLED,    HIGH);
    digitalWrite(greenLED,  LOW);
    digitalWrite(buzzerPin, HIGH);
  } else {
    // SAFE STATE
    digitalWrite(redLED,    LOW);
    digitalWrite(greenLED,  HIGH);
    digitalWrite(buzzerPin, LOW);
  }
  digitalWrite(buzzerPin, LOW);
  Serial.println("----------------------------------------");
  delay(100); // Check every 500ms
}
```

7. INPUT

The inputs to the Fire Detection and Alert System are the sensor signals produced by the IR Flame Sensor and the MQ-2 Gas/Smoke Sensor. These inputs form the basis of the decision logic.

7.1 IR Flame Sensor Input (Digital)

- Pin D7: Digital input from the IR Flame Sensor. Outputs LOW (0) when an open flame is detected within range, and HIGH (1) under normal conditions.
- The sensor detects infrared radiation emitted by flames in the 760 nm–1100 nm wavelength range.

7.2 MQ-2 Gas Sensor Input (Analog)

- Pin A0: Analog input from the MQ-2 sensor. Value ranges from 0 (clean air) to 1023 (maximum gas concentration), based on the Arduino's 10-bit ADC.
- The sensor detects LPG, propane, methane, and smoke particles. Values above 300 are treated as hazardous.

7.3 Sample Input Values

Parameter	Fire Condition	Normal Condition
Flame Sensor Value (Digital)	0 (Fire Detected)	1 (No Fire)
MQ-2 Analog Value	720 (High Smoke)	120 (Clean Air)
Action Triggered	Buzzer ON, LED ON	Buzzer OFF, LED OFF

The Serial Monitor in the Arduino IDE is used to observe both flame and smoke sensor values in real time during testing. The threshold value of 300 analog units for the MQ-2 sensor ensures the alarm is not triggered by minor environmental gas variations.

8. OUTPUT

The outputs of the Fire Detection and Alert System are the buzzer alarm, LED indicator, LCD display messages, and Serial Monitor debug output.

8.1 Buzzer and LED

- When fire or smoke is detected, the buzzer emits a continuous tone and the red LED turns on immediately.
- Both outputs are deactivated as soon as sensor readings return to normal levels.

8.2 LCD Display Output

- Normal State — Line 1: 'System Normal', Line 2: 'No Fire/Smoke'
- Flame Detected — Line 1: 'FIRE DETECTED!', Line 2: 'EVACUATE NOW!'
- Smoke Detected — Line 1: 'SMOKE DETECTED!', Line 2: 'Check Area!'

8.3 Serial Monitor Output (Sample)

```
Flame: 1 | Smoke: 115   -> Normal
Flame: 1 | Smoke: 340   -> SMOKE DETECTED!
Flame: 0 | Smoke: 210   -> FIRE DETECTED!
Flame: 1 | Smoke: 98    -> Normal
```

9. CONCLUSION

The Fire Detection and Alert System project was successfully designed, implemented, and tested using an Arduino Uno microcontroller, an IR Flame Sensor, an MQ-2 Gas/Smoke Sensor, a buzzer, LED, and 16×2 LCD display. The system demonstrated reliable real-time detection of both open flames and smoke, with immediate audio-visual alerts and informative LCD messages, validating the core concept of multi-sensor fire safety.

The threshold-based control algorithm effectively prevented false alarms from minor environmental gas fluctuations while maintaining high sensitivity to genuine fire events. The Serial Monitor confirmed that sensor readings, decision logic, and output activations all functioned as expected across multiple test conditions including a lighter flame and candle smoke.

Key learnings from this project include: digital and analog sensor interfacing, I2C LCD communication, threshold-based event logic, and embedded C programming for real-time safety applications. These skills are fundamental to IoT system design and smart environment monitoring.

10. FUTURE SCOPE

The current implementation provides a solid foundation that can be extended in several directions:

- IoT Connectivity: Integrating an ESP8266 or ESP32 Wi-Fi module to send real-time alerts via MQTT or HTTP to a cloud dashboard or mobile application, enabling remote monitoring and SMS/email notifications.
- Temperature Sensor Integration: Adding a DHT11 or DHT22 temperature and humidity sensor to detect abnormal heat rise as an additional fire precursor, improving detection reliability.
- Automatic Sprinkler Trigger: Connecting the system to a relay module that activates a water pump or sprinkler when fire is detected, enabling an automated suppression response.
- GPS Location Tagging: Incorporating a GPS module to transmit the exact geographic location of the fire alarm to emergency services via a GSM module.
- Multiple Zone Detection: Expanding the system with additional sensor nodes to cover multiple rooms or zones, with each zone independently monitored by a central Arduino or ESP32.
- Machine Learning Anomaly Detection: Implementing an edge ML model trained on sensor time-series data to distinguish genuine fire events from cooking smoke, steam, or sensor drift, reducing false positive rates.
- Battery Backup and Weatherproofing: Designing an enclosure with UPS battery backup to ensure operation during power outages, and an IP55-rated casing for industrial or outdoor deployment.

11. REFERENCES

- Arduino Documentation. LiquidCrystal_I2C Library Reference. https://www.arduino.cc/reference/en/libraries/liquidcrystal-i2c/
- Handson Technology. MQ-2 Gas Sensor Module Datasheet and User Guide. https://www.handsontec.com/dataspecs/sensor/MQ-2.pdf
- Paul Scherz and Simon Monk. Practical Electronics for Inventors. 4th Edition. McGraw-Hill Education, 2016.
- Massimo Banzi and Michael Shiloh. Getting Started with Arduino. 3rd Edition. Maker Media, 2014.
- Cytron Technologies. IR Flame Sensor Module Datasheet. https://www.cytron.io
- Texas Instruments. I2C Bus Specification and User Manual. https://www.ti.com
- R. Ramakrishnan and S. Priya. Design of IoT-Based Fire Detection System Using Arduino. International Journal of Engineering and Technology, Vol. 7, 2018.
- Vardhaman College of Engineering. IOT Laboratory Manual. Department of CSE, 2025-26.